

AI453 — Practical #3: Distributions, Sampling and Maximum Likelihood

Department of Artificial Intelligence, SVNIT Surat · B.Tech. AI, Semester VII

This will not be assessed. Plain Python 3, standard library only, nothing to install. Run `lab3_distributions.py`; it prints one passing check. The rest is yours.

The data — both printed here in full

1. **Twenty flips of a coin**, in order, 1 = heads. Six heads in twenty. In the file this is the string COIN.

1 1 0 0 0 1 0 0 1 0 0 0 0 0 1 0 1 0 0 0

2. **A loaded six-sided die**, the list DIE in the file. Sampling returns a **0-based index**, so face 1 is 0 and face 6 is 5.

face	1	2	3	4	5	6
<i>P</i>	0.25	0.20	0.20	0.15	0.18	0.02

Note the last column. It is 0.02 for a reason, and you find out why in T8.

Four helpers are written for you — `mean_of(xs)`, `var_of(xs)`, `ascii_hist(values)` and `check(label, got, want)` — along with the worked example every sampler below is built from:

```
def sample_bernoulli(p):
    return 1 if random.random() < p else 0
```

Part A — sampling

- T1. `sample_die()`** — one roll of the die above, as an index 0..5. Use the **inverse CDF**, which is four lines: draw u in $[0, 1)$, add up `DIE[0], DIE[1], ...` keeping a running total, and return the index at which the total first goes past u . Picture the line from 0 to 1 cut into six pieces of those lengths — which piece did u land in? Then draw 10000 rolls and check the six fractions against DIE.
- T2. `sample_binomial(n, p)`** — the number of 1s in n Bernoulli draws. No factorials: call `sample_bernoulli(p)` n times and add up the result. Draw 10000 values of `sample_binomial(10, 0.3)`; the mean should be near $np = 3$ and the variance near $np(1 - p) = 2.1$.

Part B — expectation and variance

- T3.** Compute the die's mean and variance **from the formula**: $E[X] = \sum_k k p_k$ and $\text{Var}(X) = \sum_k k^2 p_k - E[X]^2$. *Do this on paper first — it is twelve multiplications.* Then estimate both from $n = 10^2, 10^3, 10^4, 10^5$ samples and print the errors.
One line: each time n rises by a factor of 100, what happens to the error?
- T4. Variance of a sum.** Draw 20000 pairs twice over — once with x and y each a separate `sample_bernoulli(0.3)`, once with y set equal to x — and estimate $\text{Var}(X+Y)$ each time. Compare both with $\text{Var}(X) + \text{Var}(Y) = 0.42$. One agrees, one does not.
What is the missing term, what is it worth here, and when does it vanish?

Part C — why Gaussians are everywhere

- T5.** For $k = 1, 5, 30, 100$: build 5000 values, each the sum of k draws of `sample_bernoulli(0.1)`, and `ascii_hist()` each one. Bernoulli(0.1) is about as far from a bell as it gets — at $k = 1$ you have two bars. Look at $k = 100$. *Hint: `sample_binomial(k, 0.1)` already is that sum.* Also print the mean and variance of your 5000 sums beside $k \times 0.1$ and $k \times 0.09$, which is what Deck 3 predicts for a sum of k independent copies.
Two lines: the shape changed completely, but those two predictions held at every k — including $k = 1$, where nothing is bell-shaped. So what did the central limit theorem actually give you: the mean, the variance, or the shape?

Part D — maximum likelihood

- T6.** `likelihood(theta, flips)`, where `flips = [int(c) for c in COIN]` — the probability of getting exactly the twenty flips above if the coin comes up heads with probability θ : that is θ for each head and $(1 - \theta)$ for each tail, all multiplied. Then try every θ from 0.00 to 1.00 in steps of 0.01 and print the best one. Compare with `(heads)/20`. *You are not being asked to trust the lecture's formula — you are checking every θ by brute force and finding the maximum where the lecture said it would be.*
- T7.** Print `likelihood(0.3, flips * 65)` — the same flips repeated, 1300 in total, which is a small dataset. Look at what comes out. Then write `log_likelihood()`, adding `math.log(...)` instead of multiplying, and print that.
One line: what happened to the first number, and why does every library in this course work in logs?
- T8.** `fit_die(rolls)` — maximum likelihood for the die is just counting: $\hat{P}(\text{face } k) = (\text{times } k \text{ came up})/(\text{total rolls})$. Check it on 10^5 rolls; you should get `DIE` back.
Now break it. Fit on just **30** rolls, 100 times over, and count how many of the 100 give face 6 a probability of **exactly 0**. Compare with 0.98^{30} , which you can do with a pen. Then take one such estimate and use it to score a *new* sequence of ten rolls in which face 6 appears once — multiply the ten probabilities. Print it.